

Practical Session

19th of November

assignment sociolinguistics



Solution Part 1

Sociolinguistics

Goal

Overarching Goal:

→ create a new and enriched version of the two reddit corpora, gen-z and ask-old-people

Methods:

pre-processing:

- tokenization
 - linguistic features
 - additional features (gen-z expression occurrences)

 - creating / processing corpus objects in convokit
-

Solution

Methods:

pre-processing:

- tokenization

```
from convokit.text_processing import TextParser

# create a tokenizer and apply it to the corpora
tokenizer = TextParser(mode="tokenize")
corpus_old_tokenized = tokenizer.transform(corpus_old)
corpus_young_tokenized = tokenizer.transform(corpus_young)
```

→ this takes long for larger dataset

→ output is stored in the 'parsed' field, add another field 'num_tokens'

```
for utt in corpus_old_tokenized.iter_utterances():
    sentences = utt.meta['parsed']
    num_tokens = sum(len(sent['toks']) for sent in sentences)
    utt.meta['num_tokens'] = num_tokens
for utt in corpus_young_tokenized.iter_utterances():
    sentences = utt.meta['parsed']
    num_tokens = sum(len(sent['toks']) for sent in sentences)
    utt.meta['num_tokens'] = num_tokens
```

Solution

Methods:

- processing / filtering

```
utterances_genz = corpus_young_tokenized.get_utterances_dataframe()
filtered_utterances_genz = utterances_genz[
    (utterances_genz['meta.num_tokens'] > 10) & (utterances_genz['meta.num_tokens'] < 350)]
```

→ filter based on newly created field

→ a new corpus object can be created with a list of utterance objects

```
utterance_objects_genz = []
for utt_id in filtered_utterances_genz.index:
    utt = corpus_young_tokenized.get_utterance(utt_id)
    utterance_objects_genz.append(utt)
corpus_genz_filtered = Corpus(utterances=utterance_objects_genz)
```

Solution

Methods:

- linguistic features

```
import spacy
nlp = spacy.load("en_core_web_sm")
# process utterances with spacy pipe
processed_utterances_genz = list(nlp.pipe(utterance_samples_genz))
```

→ use spacy to process list of utterances
result is a list of spacy documents

→ use lftk Extractor object to extract linguistic features. The Extractor required spacy documents as input. Feature extraction is done under the hood (here the library uses additional information from spacy, e.g. pos-tags to calculate feature values)

```
import lftk
LFTK = lftk.Extractor(docs=processed_utterances_genz)
print("LFTK initialized for Gen Z utterances.")
features_genz_sample = LFTK.extract(features=features_to_extract)
```

→ for a larger dataset we want to do
that on a server

Solution

Methods:

- linguistic features - normalization

```
for utt in corpus_genz_filtered.iter_utterances():
    utt_id = utt.id
    features_utterance = features_genz.loc[utt_id]

    for feature in features_genz.columns:
        value = features_utterance[feature]
        # normalize if needed
        if feature.startswith("n_") and not feature.startswith("n_u"):
            value = value / utt.meta['num_tokens']
        elif feature.startswith("n_u"):
            corresponding_n_feature = feature.replace("n_u", "n_")
            n_value = features_utterance[corresponding_n_feature]
            if n_value > 0:
                value = value / n_value
            else:
                value = 0.0
        utt.meta[feature] = value
```

→ iterate over utterances. locate the corresponding feature values for each feature based on the given dataframe index and utterance id. normalize the value

Solution

Methods:

- additional features

```
# load genz wordlist
with open("/Users/falkne/PycharmProjects/nlpforSocialInteractions/additional_data/gen_z_wordlist.txt", "r") as f:
    genz_wordlist = f.read().splitlines()
print("First 10 words in the genz wordlist:", genz_wordlist[:10])
```

```
# function to count genz expressions in an utterance
def count_gen_z_words(utterance):
    tokens = utterance.meta['parsed'][0]['toks']
    tokens = [t["tok"].lower() for t in tokens]
    count_unigrams = sum(1 for t in tokens if t in genz_wordlist)
    full_string = " ".join(tokens)
    count_multigrams = 0
    for gen_z_word in genz_wordlist:
        if " " in gen_z_word:
            count_multigrams += full_string.count(gen_z_word)
    return count_unigrams + count_multigrams
```

→simplified method to count the occurrences of uni- and n-grams. Many possibilities here...

Expected Output

```
Sample utterance meta information after adding features: ConvoKitMeta({'score': 1, 'top_level_comment': 'e29tjvh',
'retrieved_on': 1537936910, 'gilded': 0, 'gildings': {'gid_1': 0, 'gid_2': 0, 'gid_3': 0}, 'subreddit': 'GenZ',
'stickied': False, 'permalink': '/r/GenZ/comments/8ybems/what_is_your_favourite_movie/e4ueb27/', 'author_flair_text':
'1998', 'parsed': [{'toks': [{'tok': 'That'}, {'tok': 'movie'}, {'tok': 'will'}, {'tok': 'break'}, {'tok': 'you'},
{'tok': '.'}]}], {'toks': [{'tok': 'Guaranteed'}, {'tok': '.'}]}], {'toks': [{'tok': 'One'}, {'tok': 'of'}, {'tok':
'the'}, {'tok': 'most'}, {'tok': 'haunting'}, {'tok': 'films'}, {'tok': 'ever'}, {'tok': 'made'}, {'tok': 'if'},
{'tok': 'you'}, {'tok': 'ask'}, {'tok': 'me'}, {'tok': '.'}]}], 'num_tokens': 21, 'num_genz_words': 0, 'n_adj':
np.float64(0.047619047619047616), 'n_adp': np.float64(0.047619047619047616), 'n_adv': np.float64(0.09523809523809523),
'n_aux': np.float64(0.047619047619047616), 'n_cconj': np.float64(0.0), 'n_det': np.float64(0.09523809523809523),
'n_intj': np.float64(0.0), 'n_noun': np.float64(0.09523809523809523), 'n_num': np.float64(0.047619047619047616),
'n_part': np.float64(0.0), 'n_pron': np.float64(0.14285714285714285), 'n_propn': np.float64(0.0), 'n_punct': np.float64(0.14285714285714285),
'n_sconj': np.float64(0.047619047619047616), 'n_sym': np.float64(0.0), 'n_verb': np.float64(0.19047619047619047),
'n_space': np.float64(0.0), 'n_uadj': np.float64(1.0), 'n_uadp': np.float64(1.0), 'n_uadv':
np.float64(0.5), 'n_uaux': np.float64(1.0), 'n_ucconj': 0.0, 'n_udet': np.float64(0.5), 'n_uintj': 0.0, 'n_unoun':
np.float64(0.5), 'n_unum': np.float64(1.0), 'n_upart': 0.0, 'n_upron': np.float64(0.6666666666666666), 'n_upropn': 0.0,
'n_upunct': np.float64(0.3333333333333333), 'n_usconj': np.float64(1.0), 'n_usym': 0.0, 'n_verb': np.float64(0.25),
'n_uspace': 0.0, 'a_word_ps': np.float64(7.0), 'a_bry_ps': np.float64(4.667), 'corr_ttr': np.float64(0.463)})
```

Sociolinguistic Analysis

Task 1:

- compare the distributions / values for each feature between the two corpora, find features that point to interesting differences
- discuss the results in a smaller group

<https://padlet.com/kbm54vknmk/results-assignment-sociolinguistics-gpai0io3rwyaub7z>

Which features show the biggest differences between the two age groups?

What do these differences tell you about the language use of younger versus older speakers?
